# 7 Algorithm design and problem solving

**In this chapter, you will learn about:**

- ★ the stages in the program development cycle:
  - analysis
  - design
  - coding
  - testing
- ★ computer systems and sub-systems
- ★ problem decomposition into component parts
- ★ methods used to design and construct solutions to problems
- ★ the purpose of an algorithm and the processes involved in it
- ★ standard methods of solution:
  - linear search
  - bubble sort
  - totalling
  - counting
  - finding average, maximum, minimum
- ★ validation checks when data is input
- ★ verification checks when data is input
- ★ use of different types of test data including:
  - documentation of a dry run using a trace table
- ★ writing, amending, identifying, and correcting errors in:
  - flowcharts
  - programs
  - pseudocode.

## 7.1 The program development life cycle

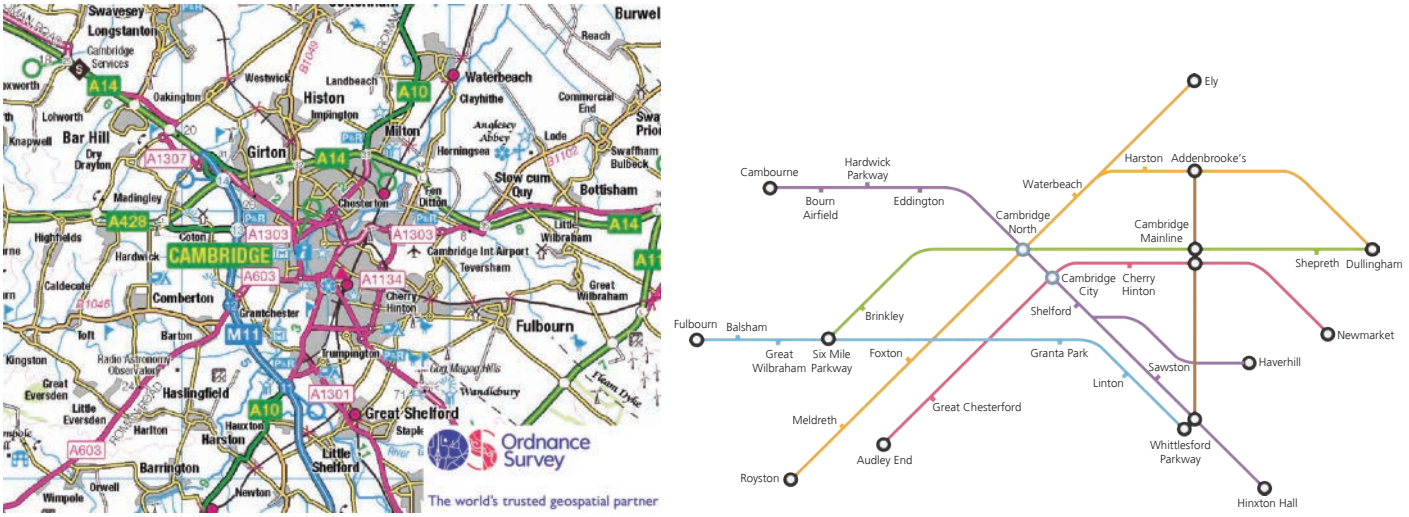
The program development life cycle is divided into five stages: analysis, design, coding, testing and maintenance. This chapter and Chapter 8 will discuss the four stages listed below:

- » **analysis**
- » **design**
- » **coding**
- » **testing**

### 7.1.1 Analysis

Before any problem can be solved, it needs to be clearly defined and set out so anyone working on the solution understands what is needed. This is called the 'requirements specification' for the program. The analysis stage uses **abstraction** and **decomposition** tools to identify exactly what is required from the program.

Abstraction keeps the key elements required for the solution to the problem and discards any unnecessary details and information that is not required. For example, a map only shows what is required for travelling from one place to another. Different methods of transport will require different types of map.




▲ **Figure 7.1** Road map and rail map

**Find out more**

Decompose getting dressed further, it can get quite complicated to show all the details required.

Decomposition breaks down a complex problem into smaller parts, which can then be subdivided into even smaller parts, that can be solved easily. Any daily task can be divided into its constituent parts.

For example, getting dressed:

- Select items to wear
- Remove any clothes being worn
- Put selected items on in order.

## 7.1.2 Design

The program specification from the analysis stage is used to show to how the program should be developed. When the design stage is complete, the programmer should know what is to be done, i.e. all the tasks that need to be completed, how each task is to be performed and how the tasks work together. This can be formally documented using **structure charts**, **flowcharts** and **pseudocode** – see Section 7.2.

## 7.1.3 Coding and iterative testing

The program or set of programs is developed. Each module of the program is written using a suitable programming language and then tested to see if it works. **Iterative testing** means that modular tests are conducted, code amended, and tests repeated until the module performs as required.

## 7.1.4 Testing

The completed program or set of programs is run many times with different sets of test data. This ensures that all the tasks completed work together as specified in the program design.

# 7.2 Computer systems, sub-systems and decomposition

A **computer system** is made up of software, data, hardware, communications and people; each computer system can be divided up into a set of sub-systems. Each sub-system can be further divided into sub-systems and so on until each sub-system just performs a single action.

Computer systems can be very large, very small or any size in between; most people interact with many different computer systems during their daily life without realising it.

**Find out more**

Find at least **five** computer systems you frequently use in your daily life; see if you can decide the size of each system.

For example, when you wake up in the morning, you might use an app on your smartphone for your alarm, then you might check the weather forecast on your computer before driving to work.

The alarm program is a very small computer system but when you check the weather forecast, you obtain the information you need from one of the largest computer systems in the world.

## 7.2.1 The computer system and its sub-systems

In order to understand how a computer system is built up and how it works it is often divided up into sub-systems. This division can be shown using top-down design to produce structure diagrams that demonstrate the modular construction of the system. Each sub-system can be developed by a programmer as a sub-routine. How each sub-routine works can be shown by using **flowcharts** or **pseudocode**.

**Top-down design** is the decomposition of a computer system into a set of sub-systems, then breaking each sub-system down into a set of smaller sub-systems, until each sub-system just performs a single action. This is an effective way of designing a computer system to provide a solution to a problem, since each part of the problem is broken down into smaller more manageable problems. The process of breaking down into smaller sub-systems is called stepwise refinement.

This structured approach works for the development of both large and small computer systems. When larger computer systems are being developed this means that several programmers can work independently to develop and test different sub-systems for the same system at the same time. This reduces the development and testing time.

## 7.2.2 Decomposing a problem

Any problem that uses a computer system for its solution needs to be decomposed into its component parts. The component parts of any computer system are:

- **inputs** – the data used by the system that needs to be entered while the system is active
- **processes** – the tasks that need to be performed using the input data and any other previously stored data
- **outputs** – information that needs to be displayed or printed for the users of the system
- **storage** – data that needs to be stored in files on an appropriate medium for use in the future.

## Example 1: An alarm app

For example, the alarm app can be decomposed into:

- inputs – time to set the alarm, remove a previously set alarm time, switch an alarm off, press snooze button
- processes – continuously check if the current time matches an alarm time that has been set, storage and removal of alarm times, management of snooze
- outputs – continuous sound/tune (at alarm time or after snooze time expired)
- storage – time(s) for alarms set.

## Activity 7.1

Using one of the computer systems that you identified, decompose it into its component parts of inputs, processes, outputs and storage.

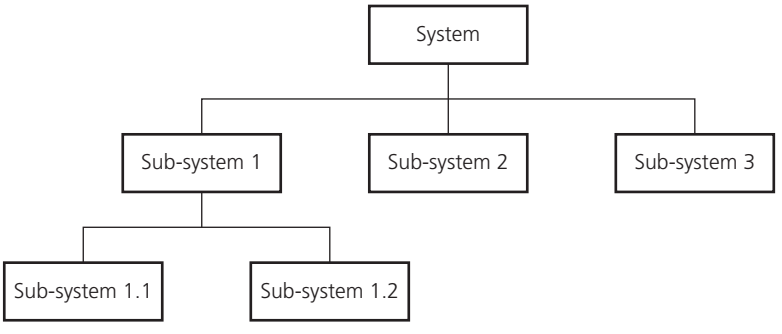
## 7.2.3 Methods used to design and construct a solution to a problem

Solutions to problems need to be designed and developed rigorously. The use of formal methods enables the process to be clearly shown for others to understand the proposed solution. The following methods need to be used by IGCSE Computer Science students:

- structure diagrams
- flowcharts
- pseudocode.

### Structure diagrams

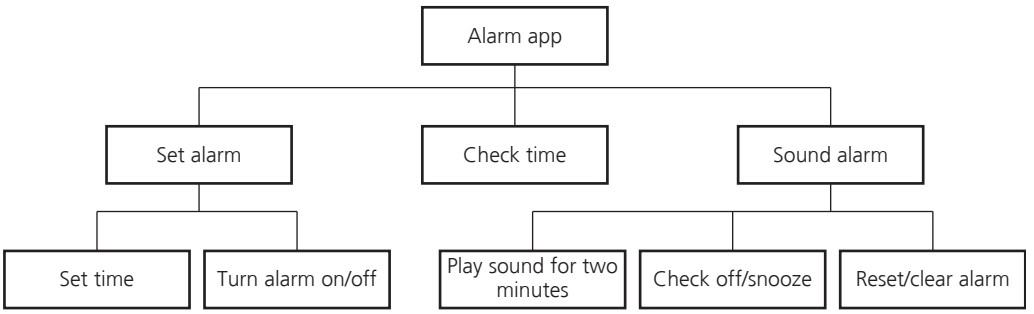
Structure diagrams can be used to show top-down design in a diagrammatic form. **Structure diagrams** are hierarchical, showing how a computer system solution can be divided into sub-systems with each level giving a more detailed breakdown. If necessary, each sub-system can be further divided.




▲ **Figure 7.2** Basic structure diagram

## Example 2: Alarm app for a smart phone

Consider the alarm app computer system for a smart phone; this could be divided into three sub-systems, setting the alarm, checking for the alarm time, sounding the alarm. These sub-systems could then be further sub-divided; a structure diagram makes the process clearer.




▲ **Figure 7.3** Structure diagram for alarm app

### Find out more

Draw a structure diagram for cleaning your teeth. If you are brave enough ask another student to try out the system to see if it works.

## Activity 7.2

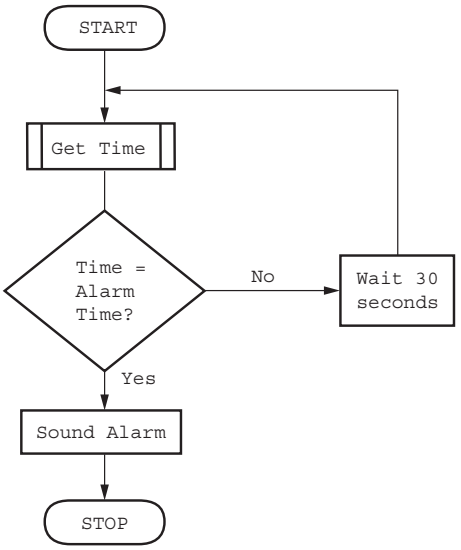
Break down the 'Check time' sub-system from the smart phone alarm app into further sub-systems.

### Flowcharts

A **flowchart** shows diagrammatically the steps required to complete a task and the order that they are to be performed. These steps, together with the order, are called an **algorithm**. Flowcharts are an effective way to communicate how the algorithm that makes up a system or sub-system works.

## Example 3: Checking for the alarm time

Have a look at a flowchart showing how the checking for the alarm time sub-system works.

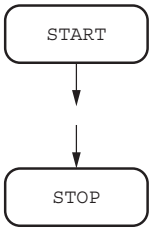



◀ **Figure 7.4** Flowchart for check time sub-system

Flowcharts are drawn using standard flowchart symbols.

### Begin/End

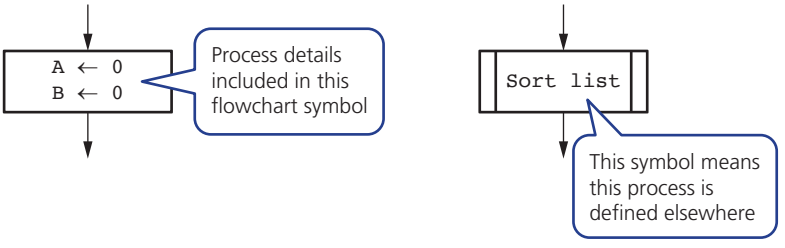
**Terminator flowchart symbols** are used at the beginning and end of each flowchart.




▲ **Figure 7.5** Terminator symbols

### Process

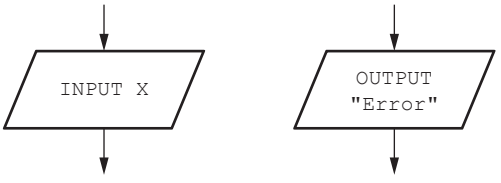
**Process flowchart symbols** are used to show actions, for example, when values are assigned to variables. If a process has been defined elsewhere then the name of that process is shown.




▲ **Figure 7.6** Process symbols

### Input and output

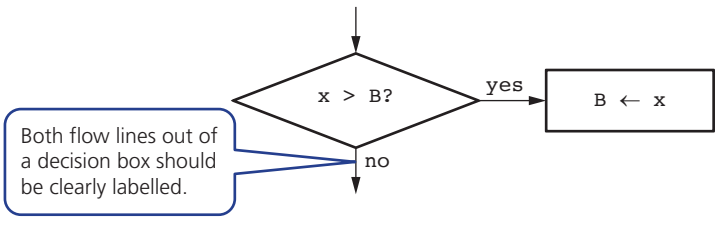
The same flowchart symbol is used to show the input of data and output of information.




▲ **Figure 7.7** Symbol used to show input and symbol used to show output

### Decision

**Decision flowchart symbols** are used to decide which action is to be taken next; these can be used for selection and repetition/iteration. There are always two outputs from a decision flowchart symbol.




▲ **Figure 7.8** Decision symbol

▲ **Figure 7.9** Flow line

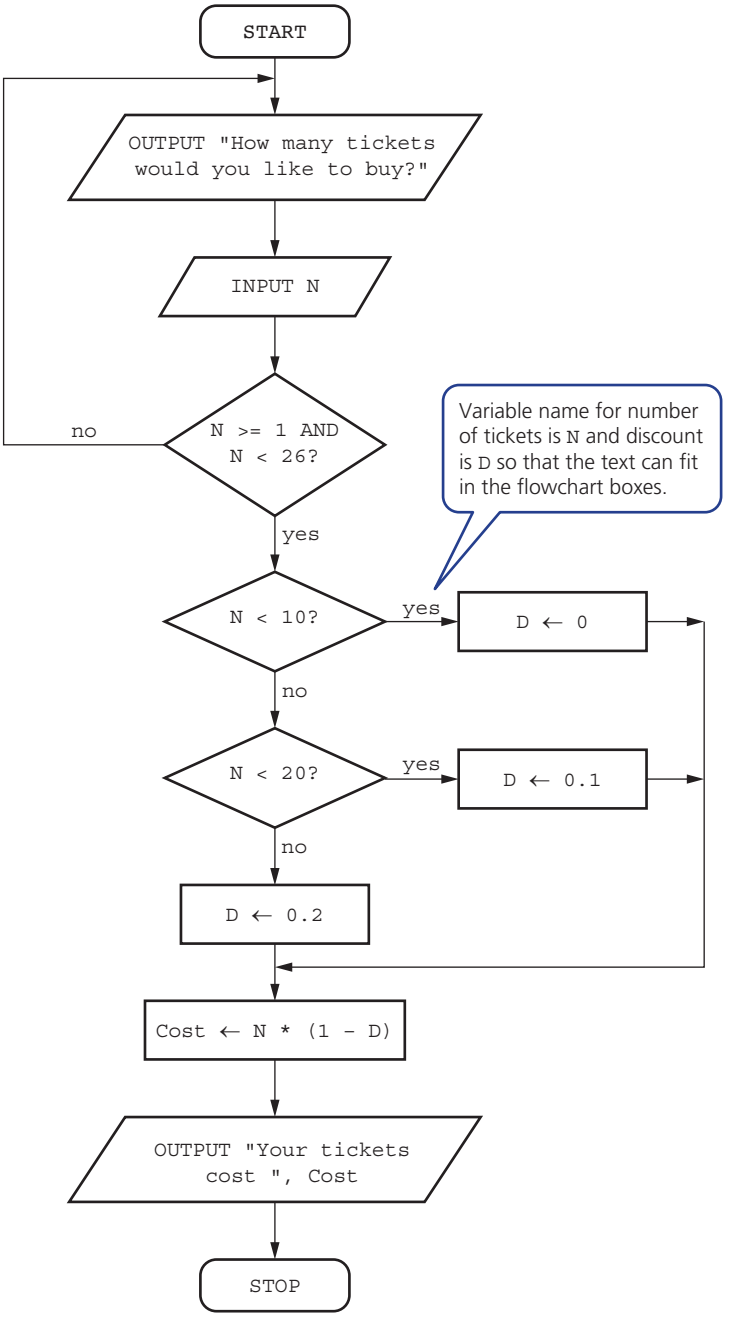
### Flow lines

**Flowchart flow lines** use arrows to show the direction of flow, which is usually, but not always, top to bottom and left to right.

## Example 4: Concert ticket sales

Tickets are sold for a concert at $20 each, if 10 tickets are bought then the discount is 10%, if 20 tickets are bought the discount is 20%. No more than 25 tickets can be bought in a single transaction.

This is flowchart showing an algorithm to calculate the cost of buying a given number of tickets:




▲ **Figure 7.10** Flowchart for ticket cost calculator

## Pseudocode

**Pseudocode** is a simple method of showing an algorithm. It describes what the algorithm does by using English key words that are very similar to those used in a high-level programming language. Data items to be processed by the algorithm are given meaningful names in the same way that variables and constants are in a high-level programming language. However, pseudocode is not bound by the strict syntax rules of a programming language. It does what its name says, it pretends to be programming code!

To ensure that pseudocode is easily understandable by others it is useful to be consistent in the way that it is written.

The pseudocode in this book is written in the following way to match the pseudocode given in the IGCSE Computer Science syllabus and to help you understand the algorithms more easily:

- a non-proportional font is used throughout
- all keywords (words used to describe a specific action e.g. INPUT) are written in capital letters
- all names given to data items and subroutines start with a capital letter
- where conditional and loop statements are used, repeated or selected statements are indented by two spaces.

### The pseudocode for an assignment statement

A value is assigned to an item/variable using the ← operator. The variable on the left of the ← is assigned the value of the expression on the right. The expression on the right can be a single value or several values combined with any of the following mathematical operators.

**Table 7.1** Mathematical operators

| Operator | Action |
|---|---|
| + | Add |
| – | Subtract |
| * | Multiply |
| / | Divide |
| ^ | Raise to the power |
| ( ) | Group |

Examples of pseudocode assignment statements:

| | |
|---|---|
| `Cost ← 10` | **Cost** has the value 10 |
| `Price ← Cost * 2` | **Price** has the value 20 |
| `Tax ← Price * 0.12` | **Tax** has the value 2.4 |
| `SellingPrice ← Price + Tax` | **SellingPrice** has the value 22.4 |
| `Gender ← "M"` | **Gender** has the value **M** |
| `Chosen ← False` | **Chosen** has the value **False** |

## Activity 7.3

What values will the following variables have after the assignments have been completed?

```
Amount ← 100
TotalPrice ← Amount * 3.5
Discount ← 0.2
FinalPrice ← TotalPrice - TotalPrice * Discount
Name ← "Nikki"
Message ← "Hello " + Name
```

### The pseudocode for conditional statements

When different actions are performed by an algorithm according to the values of the variables, conditional statements can be used to decide which action should be taken.

There are two types of conditional statement:

1 a condition that can be true or false such as: **IF … THEN … ELSE … ENDIF**

```
IF Age < 18
  THEN
    OUTPUT "Child"
  ELSE
    OUTPUT "Adult"
ENDIF
```

2 a choice between several different values, such as: **CASE OF … OTHERWISE … ENDCASE**

```
CASE OF Grade
  "A" : OUTPUT "Excellent"
  "B" : OUTPUT "Good"
  "C" : OUTPUT "Average"
  OTHERWISE OUTPUT "Improvement is needed"
ENDCASE
```

### IF … THEN … ELSE … ENDIF

For an **IF** condition the **THEN** path is followed if the condition is true and the **ELSE** path is followed if the condition is false. There may or may not be an **ELSE** path. The end of the statement is shown by **ENDIF**.

There are different ways that an **IF** condition can be set up:

- use of a Boolean variable that can have the value **TRUE** or **FALSE** (see Chapter 8 for details of Boolean variables). For example:

```
IF Found
  THEN
    OUTPUT "Your search was successful"
  ELSE
    OUTPUT "Your search was unsuccessful"
ENDIF
```

- comparisons made by using comparison operators, where comparisons are made from left to right, for example: **A > B** means '**A** is greater than **B**' Comparisons can be simple or more complicated, for example:

```
IF ((Height > 1) OR (Weight > 20)) AND (Age < 70) AND
  (Age > 5)
  THEN
    OUTPUT "You can ride"
  ELSE
    OUTPUT "Too small, too young or too old"
ENDIF
```

**Table 7.2** Comparison operators

| Operator | Comparison |
|---|---|
| > | Greater than |
| < | Less than |
| = | Equal |
| >= | Greater than or equal |
| <= | Less than or equal |
| <> | Not equal |
| **AND** | Both |
| **OR** | Either |
| **NOT** | Not |

Have a look at the algorithm below that checks if a percentage mark is valid and whether it is a pass or a fail. This makes use of two **IF** statements; the second **IF** statement is part of the first **ELSE** path. This is called a **nested IF**.

**Find out more**

Programming is covered in Chapter 8. When you have started your programming, write and test a program for this algorithm.

```
OUTPUT "Please enter a mark "
INPUT PercentageMark
IF PercentageMark < 0 OR PercentageMark > 100
  THEN
    OUTPUT "Invalid Mark"
  ELSE
    IF PercentageMark > 49
      THEN
        OUTPUT "Pass"
      ELSE
        OUTPUT "Fail"
    ENDIF
ENDIF
```

A rejected percentage mark must be either less than zero or greater than 100

This is a nested IF statement, shown clearly by the use of a second level of indentation. The percentage mark is only tested if it is in the correct range

## Activity 7.4

Re-write the algorithm to check for a mark between 0 and 20 and a pass mark of 10.

### CASE OF ... OTHERWISE ... ENDCASE

For a **CASE** statement the value of the variable decides the path to be taken. Several values are usually specified. **OTHERWISE** is the path taken for all other values. The end of the statement is shown by **ENDCASE**.

Have a look at the algorithm below that specifies what happens if the value of **Choice** is **1**, **2**, **3** or **4**.

```
CASE OF Choice
  1 : Answer ← Num1 + Num2
  2 : Answer ← Num1 - Num2
  3 : Answer ← Num1 * Num2
  4 : Answer ← Num1 / Num2
  OTHERWISE OUTPUT "Please enter a valid choice"
ENDCASE
```

## Activity 7.5

Use a **CASE** statement to display the day of the week if the variable **DAY** has a whole number value between **1** and **7** inclusive and an error message otherwise.

### The pseudocode for iteration

When some actions performed as part of an algorithm need repeating this is called iteration. Loop structures are used to perform the iteration.

Pseudocode includes these three different types of loop structure:

| | |
|---|---|
| A set number of repetitions | **FOR … TO … NEXT** |
| A repetition, where the number of repeats is not known, that is completed at least once: | **REPEAT … UNTIL** |
| A repetition, where the number of repeats is not known, that may never be completed: | **WHILE … DO … ENDWHILE** |

All types of loops can all perform the same task, for example displaying ten stars:

```
FOR Counter ← 1 TO 10
    OUTPUT "*"
NEXT Counter
```

A FOR … NEXT loop

```
Counter ← 0
REPEAT
    OUTPUT "*"
    Counter ← Counter + 1
UNTIL Counter > 9
```

A REPEAT … UNTIL loop

```
Counter ← 0
WHILE Counter < 10 DO
    OUTPUT "*"
    Counter ← Counter + 1
ENDWHILE
```

A WHILE … DO … ENDWHILE loop

As you can see, the **FOR … TO … NEXT** loop is the most efficient way for a programmer to write this type of task as the loop counter is automatically managed.

**FOR … TO … NEXT loops**

A variable is set up, with a start value and an end value, this variable is incremented in steps of one until the end value is reached and the iteration finishes. The variable can be used within the loop so long as its value is not changed. This type of loop is very useful for reading values into lists with a known length.

**Link**

For more on arrays see Chapter 8.

```
FOR Counter ← 1 TO 10
    OUTPUT "Enter Name of Student "
    INPUT StudentName[Counter]
NEXT
```

Counter starts at 1 and finishes at 10

Array (see Chapter 8) items StudentName[1] to StudentName[10] have data input

### REPEAT ... UNTIL loop

This loop structure is used when the number of repetitions/iterations is not known and the actions are repeated **UNTIL** a given condition becomes true. The actions in this loop are always completed at least once. This is a post-condition loop as the test for exiting the loop is at the end of the loop.

```
Total ← 0
Mark ← 0
REPEAT
    Total ← Total + Mark
    OUTPUT "Enter value for mark, -1 to finish "
    INPUT Mark
UNTIL Mark = -1
```

Variables `Total` and `Mark` are both initialised to zero.

At least one mark is entered.

### WHILE ... DO ... ENDWHILE loop

This loop structure is used when the number of repetitions/iterations is not known and the actions are only repeated **WHILE** a given condition is true. If the **WHILE** condition is untrue then the actions in this loop are never performed. This is a pre-condition loop as the test for exiting the loop is at the beginning of the loop.

```
Total ← 0
OUTPUT "Enter value for mark, -1 to finish "
INPUT Mark
WHILE Mark <> -1 DO
    Total ← Total + Mark
    OUTPUT "Enter value for mark, -1 to finish"
    INPUT Mark
ENDWHILE
```

Only the variable `Total` is initialised to zero

Condition tested at start of loop

### The pseudocode for input and output statements

**INPUT** and **OUTPUT** are used for the entry of data and display of information. Sometimes **READ** can be used instead of **INPUT** but this is usually used for reading from files – see Chapter 8. Also, **PRINT** can be used instead of **OUTPUT** if a hard copy is required.

**INPUT** is used for data entry; it is usually followed by a variable where the data input is stored, for example:

```
INPUT Name
```

```
INPUT StudentMark
```

**OUTPUT** is used to display information either on a screen or printed on paper; it is usually followed by a single value that is a string or a variable, or a list of values separated by commas, for example:

```
OUTPUT Name
```

```
OUTPUT "Your name is ", Name
```

```
OUTPUT Name1, "Ali", Name3
```

# 7.3 Explaining the purpose of an algorithm

An algorithm sets out the steps to complete a given task. This is usually shown as a flowchart or pseudocode, so that the purpose of the task and the processes needed to complete it are clear to those who study it.

You will be able to practise this skill as you become more familiar with writing and finding and correcting errors in algorithms.

## Example 1: Output an alarm sound

The purpose of the following pseudocode is to output the alarm sound at the appropriate time. The processes are: waiting 10 seconds, getting the current time, checking the current time with the alarm time, and outputting the alarm sound when the times match.

```
REPEAT
    Wait (10)
    Get (Time)
UNTIL Time = Alarm_Time
OUTPUT AlarmSound
```

Waits for 10 seconds

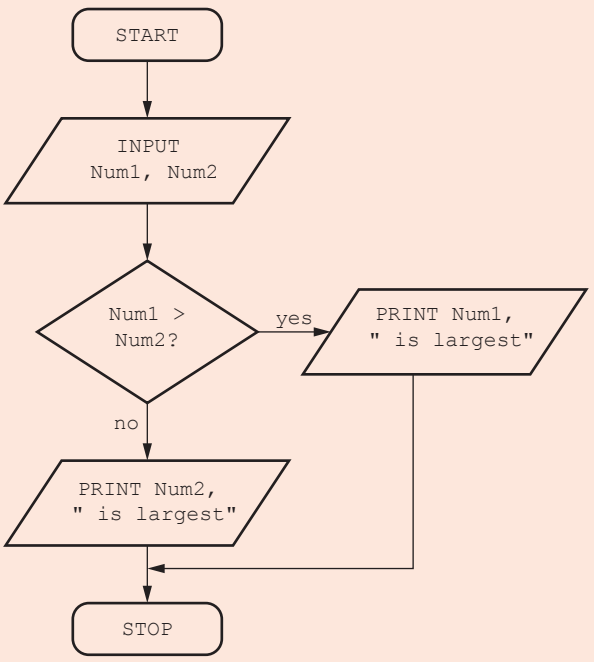
Get current time from the system clock

## Activity 7.6

Have a look at the flowchart and pseudocode below:

- identify the purpose of the algorithm that they both represent
- identify the processes included in the algorithm.

What would be output if the numbers 7 and 18 were input?




```
INPUT Num1, Num2
IF Num1 > Num2
    THEN PRINT NUM1, " is largest"
    ELSE PRINT NUM2, " is largest"
ENDIF
```

▲ **Figure 7.11** Flowchart and pseudocode

# 7.4 Standard methods of solution

The ability to repeat existing methods is very important in the design of algorithms; when an algorithm is turned into a program the same methods may be repeated many thousands of times.

You need to be able to use and understand these standard methods used in algorithms:

- Totalling
- Counting
- Finding maximum, minimum, and average (mean) values
- Searching using a linear search
- Sorting using a bubble sort.

All the standard methods of solution are shown as pseudocode algorithms and will be used to practise program writing in the next chapter.

## 7.4.1 Totalling

Totalling means keeping a total that values are added to. For example, keeping a running total of the marks awarded to each student in a class.

```
Total ← 0
FOR Counter ← 1 TO ClassSize
    Total ← Total + StudentMark[Counter]
NEXT Counter
```

Initialising `Total` to zero

Totalling the marks in an array called `StudentMark`

## 7.4.2 Counting

Keeping a count of the number of times an action is performed is another standard method. For example, counting the number of students that were awarded a pass mark:

```
PassCount ← 0
FOR Counter ← 1 TO ClassSize
    INPUT StudentMark
    IF StudentMark > 50
        THEN
            PassCount ← PassCount + 1
NEXT Counter
Count ← Count + 1
```

Initialising `PassCount` to zero

Counting the number of passes

Counting is also used to count down until a certain value is reached, for example, checking the number of items in stock in a supermarket:

```
:
NumberInStock ← NumberInStock - 1
IF NumberInStock < 20
    THEN
        CALL Reorder()
:
```

Counting down items in stock

## 7.4.3 Maximum, minimum and average

Finding the largest and smallest values in a list are two standard methods that are frequently found in algorithms, for example, finding the highest and lowest mark awarded to a class of students.

```
MaximumMark ← 0
MinimumMark ← 100
FOR Counter ← 1 TO ClassSize
  IF StudentMark[Counter] > MaximumMark
    THEN
      MaximumMark ← StudentMark[Counter]
  ENDIF
  IF StudentMark[Counter] < MinimumMark
    THEN
      MinimumMark ← StudentMark[Counter]
  ENDIF
NEXT Counter
```

Initialising maximum to the lowest mark possible

Initialising minimum to the highest possible

Calls data from an array (see Chapter 8) called `StudentMark`)

Replacing the maximum mark with a higher mark

Replacing the minimum mark with a lower mark

If the largest and smallest values are not known, an alternative method is to set the maximum and minimum values to the first item in the list.

For example, using this method to find the highest and lowest mark awarded to a class of students.

```
MaximumMark ← StudentMark[1]
MinimumMark ← StudentMark[1]
FOR Counter ← 2 TO ClassSize
  IF StudentMark[Counter] > MaximumMark
    THEN
      MaximumMark ← StudentMark[Counter]
  ENDIF
  IF StudentMark[Counter] < MinimumMark
    THEN
      MinimumMark ← StudentMark[Counter]
  ENDIF
NEXT Counter
```

Initialising minimum and maximum to the first mark

Starting the loop at the second position in the list.

Calculating the average (mean) of all the values in a list is an extension of the totalling method, for example, calculating the average mark for a class of students.

```
Total ← 0
FOR Counter ← 1 TO ClassSize
  Total ← Total + StudentMark[Counter]
NEXT Counter
Average ← Total / ClassSize
```

Calculating the average from the total after the loop has been completed

## 7.4.4 Linear search

A search is used to check if a value is stored in a list, performed by systematically working through the items in the list. There are several standard search methods, but you only need to understand one method for IGCSE Computer Science. This is called a **linear search**, which inspects each item in a list in turn to see if the item matches the value searched for.

For example, searching for a name in a class list of student names, where all the names stored are different:

```
OUTPUT "Please enter name to find "
INPUT Name
Found ← FALSE
Counter ← 1
REPEAT
  IF Name = StudentName[Counter]
```

Setting a variable, `Found`, as a flag, using `TRUE` and `FALSE` to indicate if the name has been found or not

Checking if the name input matches a name in the list

```
  THEN
     Found ← TRUE
    ELSE
      Counter ← Counter + 1
  ENDIF
UNTIL Found OR Counter > ClassSize
IF Found
  THEN
    OUTPUT Name, " found at position ",
Counter, " in the list."
  ELSE
    OUTPUT Name, " not found."
ENDIF
```

Setting the flag to TRUE when a match is found

Stopping the search when a match is found or the whole list has been searched

Outputting the position in the list when a match is found

In this example, the search checks how many people chose ice cream as their favourite dessert, where several values in the list can be the same.

```
ChoiceCount ← 0
FOR Counter ← 1 TO Length
IF "ice cream" = Dessert[Counter]
  THEN
   ChoiceCount ← ChoiceCount + 1
NEXT Counter
OUTPUT ChoiceCount, " chose ice cream
as their favourite dessert."
```

Checking ice cream has been chosen

Checking every item in the list

## 7.4.5 Bubble sort

Lists can be more useful if the items are sorted in a meaningful order. For example, names could be sorted in alphabetical order, or temperatures could be sorted in ascending or descending order. There are several standard sorting methods available, but you only need to understand one method for IGCSE Computer Science.

This method of sorting is called a **bubble sort**. Each element is compared with the next element and swapped if the elements are in the wrong order, starting from the first element and finishing with next-to-last element. Once it reaches the end of the list, we can be sure that the last element is now in the correct place. However, other items in the list may still be out of order. Each element in the list is compared again apart from the last one because we know the final element is in the correct place. This continues to repeat until there is only one element left to check or no swaps are made.

For example, the bubble sort algorithm can be used to sort a list of ten temperatures stored in the array, **`Temperature[]`**, into ascending order. It could be written in pseudocode as:

```
First ← 1
Last ← 10
REPEAT
  Swap ← FALSE
  FOR Index ← First TO Last - 1
    IF Temperature[Index] > temperature[Index + 1]
      THEN
        Temp ← Temperature[Index]
        Temperature[Index] ← Temperature[Index + 1]
        Temperature[Index + 1] ← Temp
        Swap ← TRUE
    ENDIF
  NEXT Index
  Last ← Last - 1
UNTIL (NOT Swap) OR Last = 1
```

The `IF..THEN` condition checks if temperatures are in ascending order and swaps them if they are not, using the Temp variable (short for temporary)

# 7.5 Validation and verification

In order for computer systems to only accept data inputs that are reasonable and accurate, every item of data needs to be examined before it is accepted by the system.

Two different methods, with very similar sounding names, are used. For data entry, validation ensures that only data that is reasonable is accepted. Verification is used to check that the data does not change as it is being entered.

## 7.5.1 Validation

**Validation** is the automated checking by a program that data is reasonable before it is accepted into a computer system. When data is validated by a computer system, if the data is rejected a message should be output explaining why the data was rejected and another opportunity given to enter the data.

There are many different types of validation checks including:

- **range checks**
- **length checks**
- **type checks**
- **presence checks**
- **format checks**
- **check digits.**

Different types of check may be used on the same piece of data; for example, an examination mark could be checked for reasonableness by using a range check, a type check and a presence check.

### Range check

A **range check** checks that the value of a number is between an upper value and a lower value. For example, checking that percentage marks are between 0 and 100 inclusive:

```
OUTPUT "Please enter the student's mark "
REPEAT
INPUT StudentMark
IF StudentMark < 0 OR StudentMark > 100
  THEN
    OUTPUT "The student's mark should be in the range
            0 to 100, please re-enter the mark "
ENDIF
UNTIL StudentMark >= 0 AND StudentMark <= 100
```

### Length check

A **length check** checks *either*:

- that data contains an exact number of characters, for example that a password must be exactly eight characters in length so that passwords with seven or fewer characters or nine or more characters would be rejected, for instance:

Password has a data type of string and LENGTH is the pseudocode operation that returns a whole number showing the number of characters in the string

```
OUTPUT "Please enter your password of eight
characters "
REPEAT
    INPUT Password
    IF LENGTH(Password) <> 8
      THEN
        OUTPUT "Your password must be exactly eight
                characters, please re-enter "
    ENDIF
UNTIL LENGTH(Password) = 8
```

- *or* that the data entered is a reasonable number of characters, for example, a family name could be between two and thirty characters inclusive so that names with one character or thirty-one or more characters would be rejected.

FamilyName has a data type of string and LENGTH is the pseudocode operation that returns a whole number showing the number of characters in the string

```
OUTPUT "Please enter your family name "
REPEAT
  INPUT FamilyName
  IF LENGTH(FamilyName) > 30 OR LENGTH(FamilyName) < 2
    THEN
      OUTPUT "Too short or too long,
      please re-enter "
  ENDIF
UNTIL LENGTH(FamilyName) <= 30 AND LENGTH(FamilyName) >= 2
```

**Link**

To understand some of the concepts and commands in this code, such as **DIV**, see Chapter 8.

**Find out more**

Programming is covered in Chapter 8. When you have started your programming, find out how you could test for a whole number and write and test a program for this validation rule.

### Type check

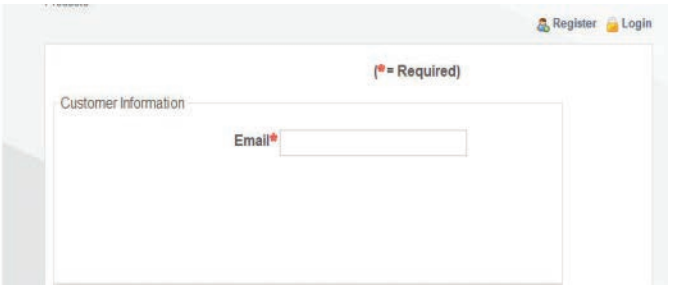
A type check checks that the data entered is of a given data type, for example, that the number of brothers or sisters would be an integer (whole number).

```
OUTPUT "How many brothers do you have? "
REPEAT
INPUT NumberOfBrothers
IF NumberOfBrothers <> DIV(NumberOfBrothers, 1)
  THEN
    OUTPUT "This must be a whole number, please re-enter"
ENDIF
UNTIL NumberOfBrothers = DIV(NumberOfBrothers, 1)
```

### Presence check

A **presence check** checks to ensure that some data has been entered and the value has not been left blank, for example, an email address for an online transaction must be completed.

```
OUTPUT "Please enter your email address "
REPEAT
INPUT EmailAddress
IF EmailAddress = ""
  THEN
   OUTPUT "*=Required "
ENDIF
UNTIL EmailAddress <> ""
```




▲ **Figure 7.12** Presence check error message

### Format check and check digit

A **format check** checks that the characters entered conform to a pre-defined pattern, for example, in Chapter 9 the cub number must be in the form CUB9999. The pseudocode for this example will be given in the string handling section of Chapter 9.

A **check digit** is the final digit included in a code; it is calculated from all the other digits in the code. Check digits are used for barcodes, product codes, International Standard Book Numbers (ISBN) and Vehicle Identification Numbers (VIN).

**Link**

ISBN and modulo-11 check digit calculations are covered in Chapter 2.

Check digits are used to identify errors in data entry caused by mis-typing or mis-scanning a barcode. They can usually detect the following types of error:

- an incorrect digit entered, for example, 5327 entered instead of 5307
- transposition errors where two numbers have changed order for example 5037 instead of 5307
- omitted or extra digits, for example, 537 instead of 5307 or 53107 instead of 5307
- phonetic errors, for example, 13, thirteen, instead of 30, thirty.




▲ **Figure 7.13** ISBN 13 code with check digit

**Find out more**

1. Find an ISBN, then show that its check digit is correct.
   Working in pairs find two ISBNs each, copy one down with a transposition error and the other one correctly.
   Swap your ISBNs and see if you can find the one with the error.
   Look at a correct ISBN, can you think of an error that this system will not identify and explain with an example why this is the case?
2. Find out how limit checks and consistency checks are used.

## Activity 7.7

1 State, with reasons, which validation checks you could use for the following inputs.

You may decide that more than one validation check is required.
- Entering a telephone number
- Entering a pupil's name
- Entering a part number in the form XXX999, when X must be a letter and 9 must be a digit.

2 Write an algorithm using pseudocode to check the age and height of a child who wants to go on a fairground ride. The age must be over 7 and under 12, the height must be over 110 centimetres and under 150 centimetres.

3 Write an algorithm using pseudocode to check that the length of a password is between 8 and 12 characters inclusive.

## 7.5.2 Verification

**Verification** is checking that data has been accurately copied from one source to another – for instance, input into a computer or transferred from one part of a computer system to another.

Verification methods for input data include:

- Double entry
- Screen/visual check.

For **double entry** the data is entered twice, sometimes by different operators. The computer system compares both entries and if they are different outputs an error message requesting that the data is entered again.




▲ **Figure 7.14** Double entry

A **screen/visual check** is a manual check completed by the user who is entering the data. When the data entry is complete the data is displayed on the screen and the user is asked to confirm that it is correct before continuing. The user either checks the data on the screen against a paper document that is being used as an input form or, confirms whether it is correct from their own knowledge.

**Link**

Parity checks and checksums are used when data is transferred from one part of a computer system to another, or across a network, and are discussed in Chapter 2.

### Activity 7.8

Explain why the following input data also needs to be verified:

- Entering a telephone number
- Entering a pupil's name
- Entering a part number in the form XXX999, when X must be a letter and 9 must be a digit.

# 7.6 Test data

## 7.6.1 How to suggest and apply suitable test data

In order to determine whether a solution is working as it should, it needs to be tested. Usually before a whole system is tested each sub-system is tested separately.

Algorithms written in pseudocode or as flowcharts can be tested by a person working through them using any data that is required and seeing what the result is. Computer programs can be tested by running them on a computer using any data that is required and seeing what result is output. However, in order to test a solution thoroughly it may need to be worked through several times with different sets of test data.

A **set of test data** is all the items of data required to work through a solution. For instance, the set of test data used for Activity 7.6 was 7 and 18.

In order to prove that program or algorithm solutions do what they are supposed to do, a set of test data should be used that the program would normally be expected to work with, together with the result(s) that are expected from that data. The type of test data used to do this is called **normal data**. Normal data should be used to work through the solution to find the actual result(s) and see if they are the same as the expected result(s).

For example, consider an algorithm that records the percentage marks, entered in whole numbers, from ten end-of-term examinations for a pupil, and then finds the average mark. A set of normal test data for this purpose could be:

Normal test data: 50, 50, 50, 50, 50, 50 50, 50, 50, 50

Expected result: 50

### Activity 7.9

Provide a more realistic set of test data and its expected result.

Solutions also need to be tested to prove that they do **not** do what they are supposed not to do. In order to do this, test data should be chosen that would be rejected by the solution as not suitable, if the solution is working properly. This type of test data is called **abnormal test data**. (It is also sometimes called **erroneous test data**.)

For example, erroneous/abnormal data for our algorithm to find the average percentage marks from ten end of term examinations could be:

Erroneous/abnormal data: -12, eleven

Expected results: both of these values should be rejected

### Activity 7.10

Provide some more erroneous/abnormal data and its expected results.

When testing algorithms with numerical values, sometimes only a given range of values should be allowed. For example, percentage marks should only be in the range 0 to 100. Our algorithm above should be tested with **extreme data**. Extreme data are the largest and smallest values that normal data can take. In this case:

Extreme data: 0, 100

Expected results: these values should be accepted

There is another type of test data called **boundary data**. This is used to establish where the largest and smallest values occur. At each boundary two values are required: one value is accepted and the other value is rejected. For example, for percentage marks in the range 0 to 100, the algorithm should be tested with the following **boundary data**:

Boundary data for 0 is: -1, 0

Expected results: -1 is rejected, 0 is accepted

### Activity 7.11

1 Provide boundary data for the upper end of the range; assume that the percentage marks are always whole numbers.

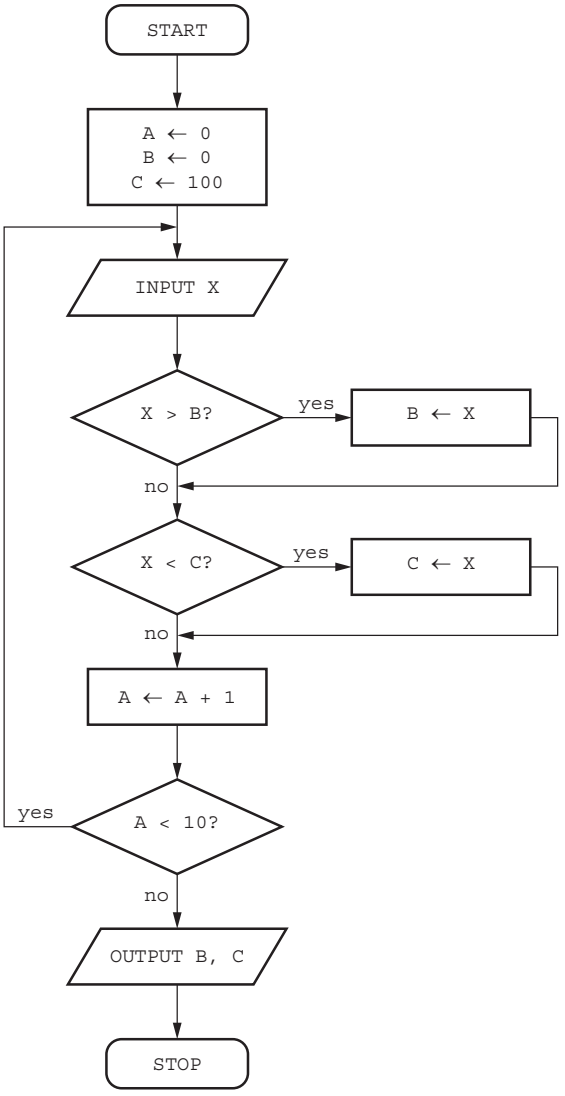
2 The end of term examinations are now marked out of twenty. Provide the following:
- Two sets of normal data and their expected results
- Some erroneous/abnormal data and their expected results
- Two sets of boundary data and their expected results.

## 7.7 Trace tables to document dry runs of algorithms

A thorough structured approach is required to find out the purpose of an algorithm. This involves recording and studying the results from each step in the algorithm and requires the use of test data.

## Worked example

Consider the algorithm represented by the following flowchart:




▲ **Figure 7.15** Flowchart to trace

A **trace table** can be used to record the results from each step in an algorithm; it is used to record the value of an item (variable) each time that it changes. The manual exercise of working through an algorithm step by step is called a **dry run**.

A trace table is set up with a column for each variable and a column for any output. For example:

▼ **Table 7.3** Trace table

| A | B | C | X | OUTPUT |
|---|---|---|---|---|
| 0 | 0 | 100 | | |
| | | | | |

Test data is then used to dry run the flowchart and record the results on the trace table. During a dry run:

- every time the value of a variable is changed, the new value is entered in that column of the trace table
- every time a value is output, the value is shown in the output column.

Test data: 9, 7, 3, 12, 6, 4, 15, 2, 8, 5

▼ **Table 7.4** Completed trace table for flowchart

| A | B | C | X | OUTPUT |
|---|---|---|---|---|
| 0 | 0 | 100 | | |
| 1 | 9 | 9 | 9 | |
| 2 | | 7 | 7 | |
| 3 | | 3 | 3 | |
| 4 | 12 | | 12 | |
| 5 | | | 6 | |
| 6 | | | 4 | |
| 7 | 15 | | 15 | |
| 8 | | 2 | 2 | |
| 9 | | | 8 | |
| 10 | | | 5 | |
| | | | | 15 2 |

The values 15 and 2 without the comma

## Activity 7.12

Use a trace table and the test data 400, 800, 190, 170, 300, 110, 600, 150, 130, 900 to record another dry run of the highest and lowest number flowchart from Section 7.7.

It can be seen from the output that the algorithm selects the largest and the smallest numbers from a list of ten positive numbers. The same trace table could have been used if the algorithm had been shown using pseudocode:

```
A ← 0
B ← 0
C ← 100
OUTPUT "Enter your ten values"
REPEAT
  INPUT X
  IF X > B
    THEN
      B ← X
  ENDIF
  IF X < C
    THEN
      C ← X
  ENDIF
  A ← A + 1
UNTIL A = 10
OUTPUT B, C
```

Often questions will use variables with a single letter, instead of a meaningful identifier, and could ask what the purpose of the algorithm is!

This prompt needs to be shown as the first output. No quotation marks should be shown in the output column

## Activity 7.13

Use the trace table below and the test data 4, 8, 19, 17, 3, 11, 6, 1, 13, 9 to record a dry run of the pseudocode.

▼ **Table 7.5** Trace table to complete for the pseudocode

| A | B | C | X | OUTPUT |
|---|---|---|---|---|
| 0 | 0 | 100 | | 10 |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |

## Activity 7.14

Draw a trace table for the bubble sort algorithm on page 276 and use the test data 35, 31, 32, 36, 39, 37, 42, 38 to record a dry run of the pseudocode.

# 7.8 Identifying errors in algorithms

Trace tables and test data can be used to identify and correct errors.

Your completed trace table for Activity 7.14 should look like this:

▼ **Table 7.6** Completed trace table for flowchart

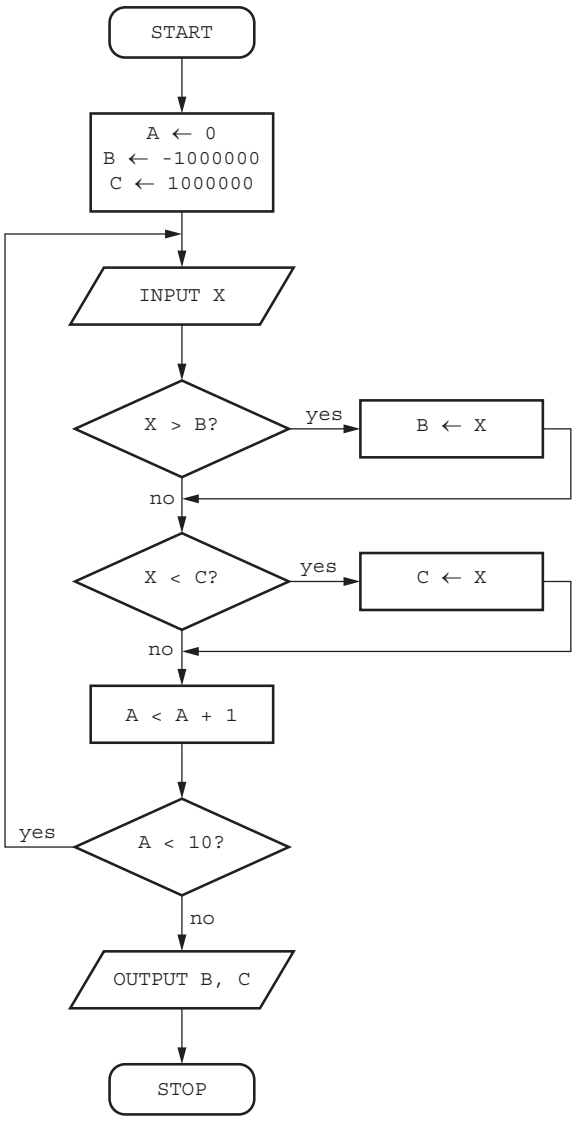
| A | B | C | X | OUTPUT |
|---|---|---|---|---|
| 0 | 0 | 100 | | |
| 1 | 400 | | 400 | |
| 2 | 800 | | 800 | |
| 3 | | | 190 | |
| 4 | | | 170 | |
| 5 | | | 300 | |
| 6 | | | 110 | |
| 7 | | | 600 | |
| 8 | | | 150 | |
| 9 | | | 130 | |
| 10 | 900 | | 900 | |
| | | | | 900 100 |

There is an error as the smallest number, 110, has not been identified.

## Activity 7.15

Use a trace table and some negative test data to record another dry run of the pseudocode or flowchart. What error have you found?

As this algorithm only works for numbers between 0 and 100; a better algorithm could look like this:



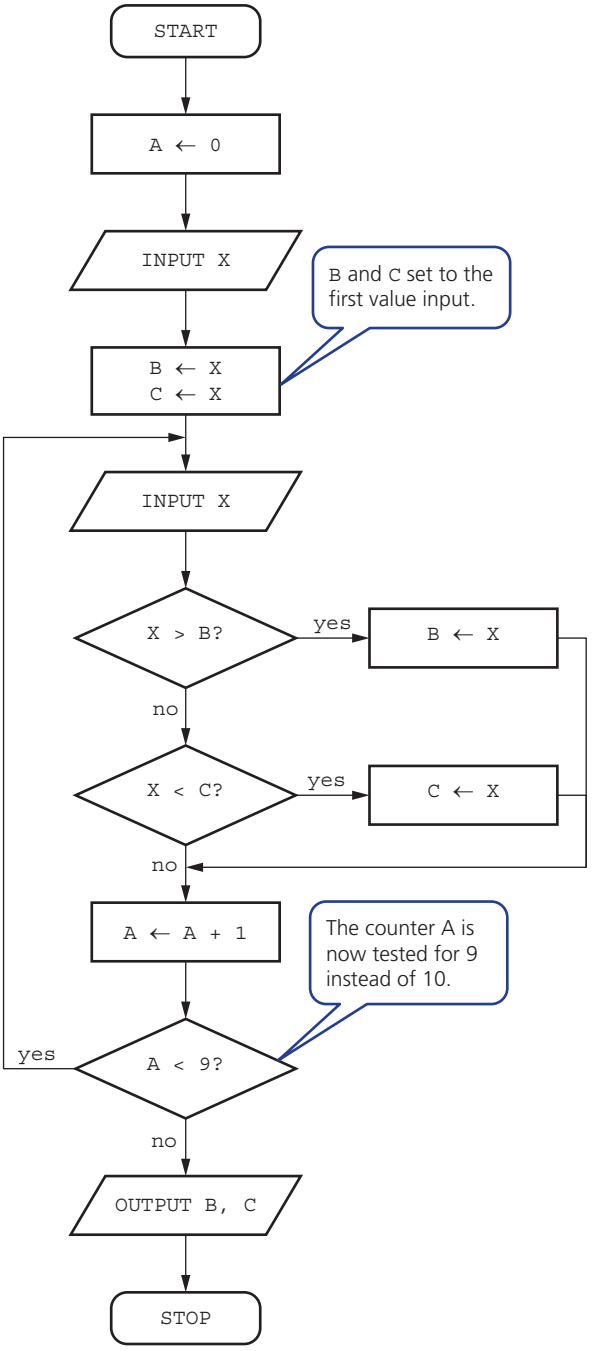

▲ **Figure 7.16** A better algorithm

This algorithm is very similar and works for a much larger range of numbers, but it still does not work for every set of numbers.

## Activity 7.16

Identify two numbers where the algorithm will still fail.

In order to work for any set of numbers, the algorithm needs to be re-written to allow the largest and smallest numbers to be tested against numbers that appear in any list provided. The provisional values set at the start of the algorithm need to be chosen from the list. A standard method is to set both these provisional values to the value of the first item input.




▲ **Figure 7.17** A much better algorithm

## Activity 7.17

Rewrite the pseudocode so it works for every set of numbers like the flowchart above.

Test your pseudocode algorithm with this set of test data:

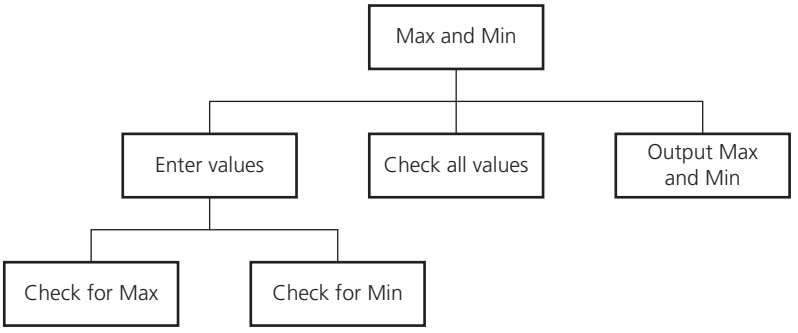
-97, 12390, 0, 77, 359, -2, -89, 5000, 21, 67

# 7.9 Writing and amending algorithms

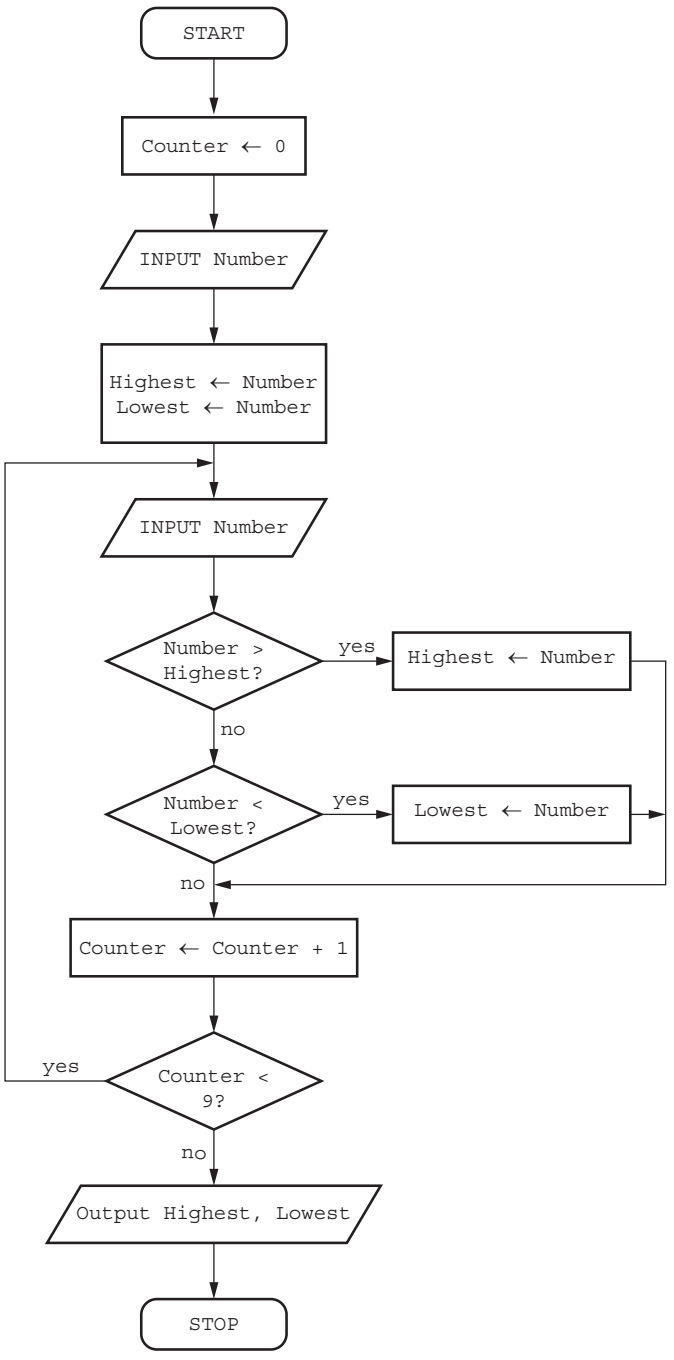
There are a number of stages when producing an algorithm for a given problem:

1. Make sure that the problem is clearly specified – the purpose of the algorithm and the tasks to be completed by the algorithm.
2. Break the problem down in to sub-problems; if it is complex, you may want to consider writing an algorithm for each sub-problem. Most problems, even the simplest ones can be divided into:
   - Set up processes
   - Input
   - Processing of data
   - Permanent storage of data (if required)
   - Output of results.
3. Decide on how any data is to be obtained and stored, what is going to happen to the data and how any results are going to be displayed.
4. Design the structure of your algorithm using a structure diagram.
5. Decide on how you are going to construct your algorithm, either using a flowchart or pseudocode. If you are told how to construct your algorithm, then follow the guidance.
6. Construct your algorithm, making sure that it can be easily read and understood by someone else. Precision is required when writing algorithms, just as it is when writing program code. This involves setting it out clearly and using meaningful names for any data stores. Take particular care with conditions used for loops and selection, for example 'Counter >= 10' rather than 'Counter ten or over'. The algorithms that you have looked at so far in this chapter were not designed with readability in mind because you needed to work out what the problem being solved was.
7. Use several sets of test data (Normal, Abnormal and Boundary) to dry run your algorithm and show the results in trace tables, to enable you to find any errors.
8. If any errors are found, correct them and repeat the process until you think that your algorithm works perfectly.

Have a look at this structure diagram and flowchart for the algorithm to select the largest, Max, and smallest, Min, numbers from a list of ten numbers. This time the flowchart is more easily readable than the structure chart:




▲ **Figure 7.18** Structure chart for Max and Min




▲ **Figure 7.19** A more easily understandable flowchart for Max and Min

## Example 1: Writing algorithms in pseudocode

Tickets are sold for a concert at $20 each. If 10 tickets are bought then the discount is 10%, if 20 tickets are bought the discount is 20%. No more than 25 tickets can be bought in a single transaction.

**a** Use pseudocode to write the algorithm to calculate the cost of buying a given number of tickets.

**b** Explain how you would test your algorithm.

**a**

```
REPEAT
  OUTPUT "How many tickets would you like to buy? "
  INPUT NumberOfTickets
UNTIL NumberOfTickets > 0 AND NumberOfTickets < 26
IF NumberOfTickets < 10
  THEN
    Discount ← 0
  ELSE
    IF NumberOfTickets < 20
      THEN
        Discount ← 0.1
      ELSE
        Discount ← 0.2
    ENDIF
ENDIF
Cost ← NumberOfTickets * 20 * (1 - Discount)
PRINT "Your tickets cost ", Cost
```

**b** I would use test data with values of:

| | |
|---|---|
| **0, 26,** | Expected results rejected |
| **1, 25,** | Expected results **20, 400** |
| **9, 10,** | Expected results **180, 180** |
| **19, 20,** | Expected results **342, 320** |

## Activity 7.18

For the test data given in Example 1, identify the type of test data used and suggest some more test data and dry run the algorithm.

## ? Example 2: Writing algorithms in pseudocode

A school with 600 students wants to produce some information from the results of the four standard tests in Maths, Science, English and IT. Each test is out of 100 marks. The information output should be the highest, lowest and average mark for each test and the highest, lowest and average mark overall. All the marks need to be input.

**a** Use pseudocode to write the algorithm to complete this task.

**b** Explain how you would test your algorithm.

**a**

Comments are used to make the algorithm more understandable

```
// initialisation of overall counters
OverallHighest ← 0
OverallLowest ← 100
OverallTotal ← 0
FOR Test ← 1 TO 4  // outer loop for the tests
```

```
// initialisation of subject counters
  SubjectHighest ← 0
  SubjectLowest ← 100
  SubjectTotal ← 0
  CASE OF Test
    1 : SubjectName ← "Maths"
    2 : SubjectName ← "Science"
    3 : SubjectName ← "English"
    4 : SubjectName ← "IT"
  ENDCASE
  FOR StudentNumber ← 1 TO 600  // inner loop for the
students
    REPEAT
      OUTPUT "Enter Student", StudentNumber, "'s mark for
", SubjectName
      INPUT Mark
    UNTIL Mark < 101 AND Mark > -1
    IF Mark < OverallLowest THEN OverallLowest ← Mark
    IF Mark < SubjectLowest THEN SubjectLowest ← Mark
    IF Mark > OverallHighest THEN OverallHighest ← Mark
    IF Mark > SubjectHighest THEN SubjectHighest ← Mark
    OverallTotal ← OverallTotal + Mark
    SubjectTotal ← SubjectTotal + Mark
  NEXT StudentNumber
  SubjectAverage ← SubjectTotal / 600
  OUTPUT SubjectName
  OUTPUT "Average mark is ", SubjectAverage
  OUTPUT "Highest Mark is ", SubjectHighest
  OUTPUT "Lowest Mark is ", SubjectLowest
NEXT Test
OverallAverage ← OverallTotal / 2400
OUTPUT "Overall Average is ", OverallAverage
OUTPUT "Overall Highest Mark is ", OverallHighest
OUTPUT "Overall Lowest Mark is ", OverallLowest
```

**b** For the algorithm to be tested by dry running, I would reduce the number of students to 5 and the number of subjects to 2.

**Find out more**

Programming is covered in Chapter 8. When you have started your programming, write and test programs for Examples 1 and 2.

More practice on writing algorithms will be given in Chapter 8.

## Activity 7.19

1 Identify the changes you would need to make to the algorithm in Example 2 to reduce the number of students to 5 and the number of subjects to 2.

Identify the test data needed to test Example 2 with the reduced number of students and subjects.

2 With the set of test data you have chosen set up, complete a trace table so that you can compare your expected results with the actual results when you dry run the algorithm.

## Activity 7.20

1 Write pseudocode to input ten positive numbers and find the total and the average.

2 Write pseudocode to input any number of positive numbers and find the total and the average. The user should enter '-1' when they have finished entering their list of positive numbers.

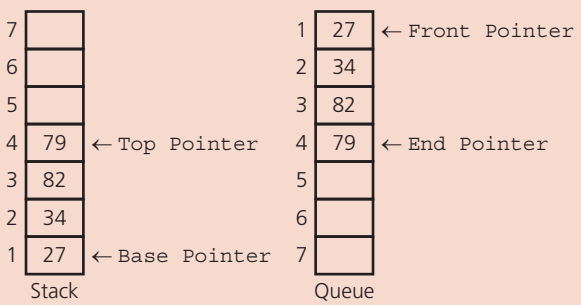
3 Explain why you chose the loop structure for each task.

## Extension

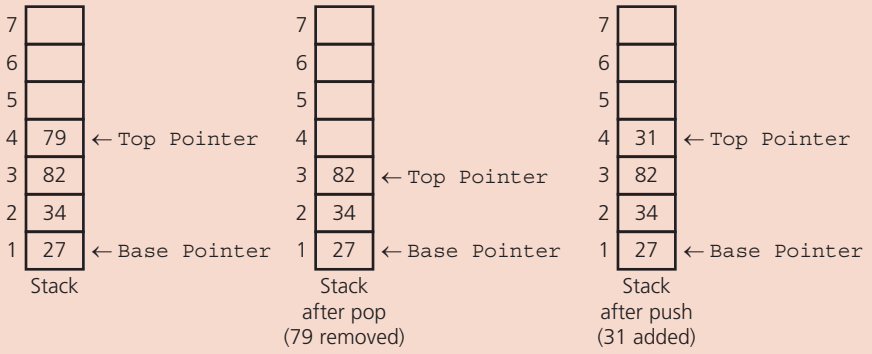
For those students interested in studying computer science at A Level, the following section is an introduction to the use of Abstract Data Types (ADTs) to store data in stacks and queues.

An **ADT** is a collection of data and a set of operations on that data. For example, a stack includes the items held on the stack and the operations to add an item to the stack (push) or remove an item from the stack (pop):

- **stack** – a list containing several items operating on the Last In First Out (LIFO) principle. Items can be added to the stack (push) and removed from the stack (pop). The first item added to a stack is the last item to be removed from the stack.
- **queue** – a list containing several items operating on the First In First Out (FIFO) principle. Items can be added to the queue (enqueue) and removed from the queue (dequeue). The first item added to a queue is the first item to be removed from the queue.

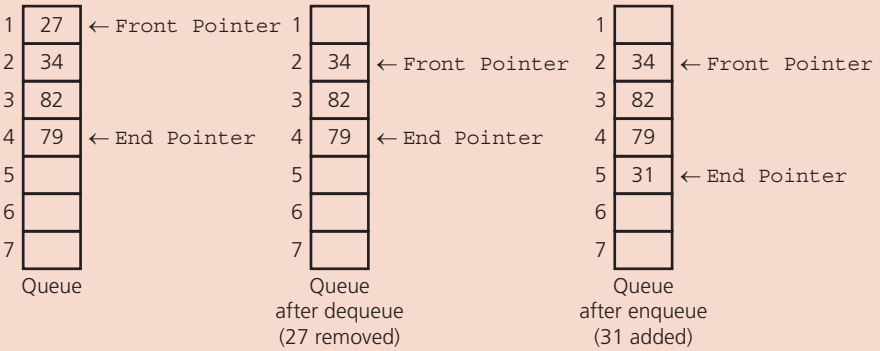



▲ **Figure 7.20** In both of these examples 27 was the first item added and 79 the last item added




▲ **Figure 7.21** Stack operations

The value of the **`Base Pointer`** always remains the same during stack operations.




▲ **Figure 7.22** Queue operations

The values of both the **`Front Pointer`** and the **`End Pointer`** change during queue operations.

### Extension activity

Show a stack and pointers after a pop operation. Show a queue and pointers after a dequeue operation.

In this chapter, you have learnt about:

- ✔ the program development life cycle
- ✔ decomposition of systems and problems in sub-systems and sub-problems
- ✔ design and construction of algorithms to solve problems using structure diagrams, flowcharts, and pseudocode
- ✔ explaining the purpose of an algorithm
- ✔ standard methods of solution
- ✔ the need and purpose of validation and verification checks on input data
- ✔ suggesting and applying suitable test data
- ✔ trace tables and dry runs
- ✔ writing, amending and identifying errors in algorithms.

## Key terms used throughout this chapter

**analysis** – part of the program development life cycle; a process of investigation, leading to the specification of what a program is required to do

**design** – part of the program development life cycle; uses the program specification from the analysis stage to show to how the program should be developed

**coding** – part of the program development life cycle; the writing of the program or suite of programs

**testing** – part of the program development life cycle; systematic checks done on a program to make sure that it works under all conditions

**abstraction** – a method used in the analysis stage of the program development life cycle; the key elements required for the solution to the problem are kept and any unnecessary details and information that are not required are discarded

**decomposition** – a method used in the analysis stage of the program development life cycle; a complex problem is broken down into smaller parts, which can then be sub divided into even smaller parts that can be solved more easily

**top-down design** – the breaking down of a computer system into a set of sub-systems, then breaking each sub-system down into a set of smaller sub-systems, until each sub-system just performs a single action

**inputs** – the data used by the system that needs to be entered while the system is active

**processes** – the tasks that need to be performed by a program using the input data and any other previously stored data

**output** – information that needs to be displayed or printed for the users of the system

**storage** – data that needs to be stored in files on an appropriate media for use in the future

**structure diagram** – a diagram that shows the design of a computer system in a hierarchical way, with each level giving a more detailed breakdown of the system into sub-systems

**flowchart** – a diagram that shows the steps required for a task (sub-system) and the order in which the steps are to be performed

**algorithm** – an ordered set of steps to solve a problem

**pseudocode** – a simple method of showing an algorithm; it describes what the algorithm does by using English key words that are very similar to those used in a high-level programming language but without the strict syntax rules

**linear search** – an algorithm that inspects each item in a list in turn to see if the item matches the value searched for

**bubble sort** – an algorithm that makes multiple passes through a list comparing each element with the next element and swapping them. This continues until there is a pass where no more swaps are made

**validation** – automated checks carried out by a program that data is reasonable before it is accepted into a computer system

**verification** – checking that data has been accurately copied from another source and input into a computer or transferred from one part of a computer system to another

**set of test data** – all the items of data required to work through a solution

**normal data** – data that is accepted by a program

**abnormal data** – data that is rejected by a program

**extreme data** – the largest/smallest data value that is accepted by a program

**boundary data** – the largest/smallest data value that is accepted by a program and the corresponding smallest/largest rejected data value

**range check** – a check that the value of a number is between an upper value and a lower value

**length check** – a method used to check that the data entered is a specific number of characters long or that the number of characters is between an upper value and a lower value

**type check** – a check that the data entered is of a specific type

**presence check** – a check that a data item has been entered

**format check** – a check that the characters entered conform to a pre-defined pattern

**check digit** – an additional digit appended to a number to check if the entered number is error-free; check digit is a data entry check and not a data transmission check

# Exam-style questions

**1** A solution to a problem is decomposed into its component parts. Name and describe the component parts. [8]

**2** A computer system is to be developed to provide a modulo 11 check digit for numbers from 4 to 20 digits in length. Provide a structure diagram for this computer system. [6]

**3** A phone app is being developed to split the cost of a restaurant bill between a given number of people. It is being designed to work for up to 12 diners and for bills from $10 to $500.

**a** What validation checks should be used for the number of diners and the size of the bill? [2]

**b** Provide two sets of normal data and their expected results. [4]

**c** Provide some abnormal/erroneous data. [1]

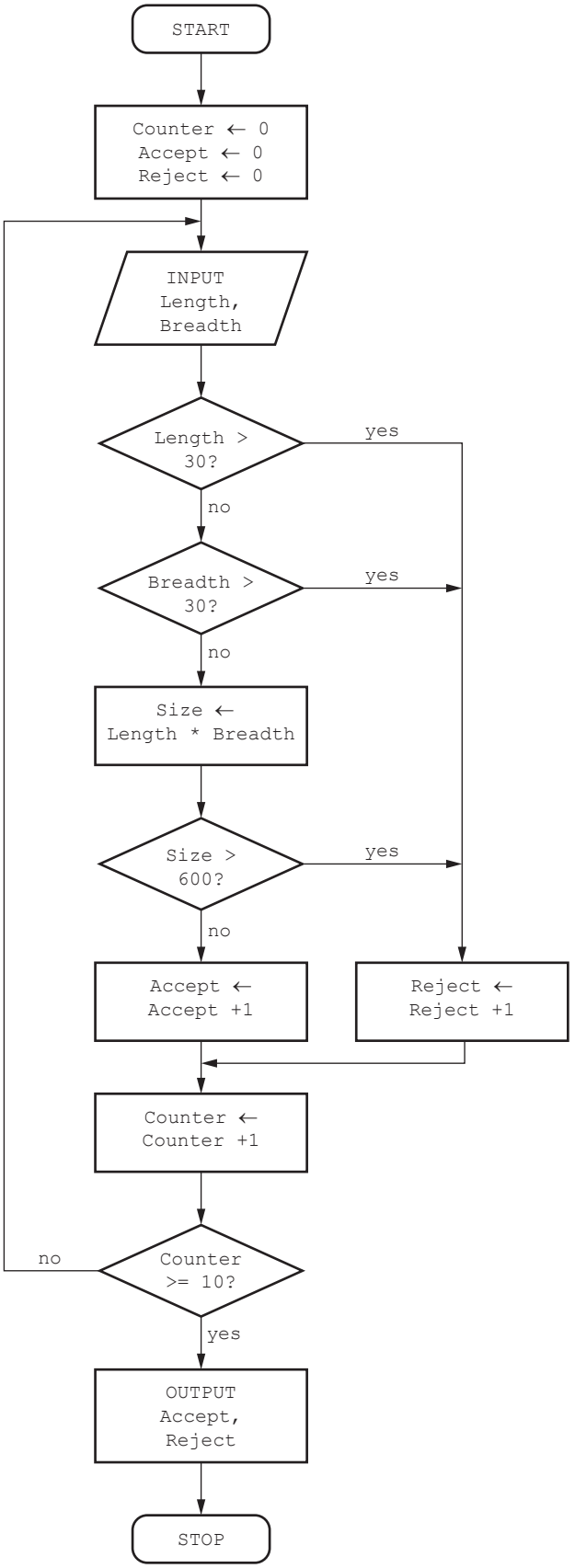
**d** Identify the boundary data required and the expected results. [4]

**4** Explain what is meant by validation and verification. [4]

**5** The following data is to be entered onto an online form:
- Name
- Date of birth
- Password
- Phone number.

For each item state, with reasons, the validation and verification checks that should be used on the input data. [8]

**6** The following algorithm, shown as a flowchart, checks the size of a consignment of ten parcels. The dimensions of each parcel are input in centimetres.



**a** Use this data and the following trace table to dry run the algorithm:
15, 10, 20, 17, 32, 10, 30, 35, 30, 15, 30, 28, 25, 25, 20, 15, 40, 20, 12, 10

| Counter | Length | Breadth | Volume | OUTPUT |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |

[5]

**b** State the processes included in this algorithm. [3]

**c** Identify the rules required to accept a parcel. [3]

**7** The following algorithm written in pseudocode adds up 10 positive numbers and outputs the total. It contains several errors.

```
Counter ← 1
FOR Counter ← 1 TO 10
  REPEAT
    OUTPUT "Enter a positive whole number "
    INPUT Number
  UNTIL Number < 0
  Total ← Total + Counter
  Counter ← Counter + 1
  OUTPUT Total
NEXT Number
```

**a** Identify all the errors in the algorithm. [5]

**b** Rewrite the algorithm so that it is effective and error free. [4]

**c** Set up a trace table and some test data to dry run your rewritten algorithm. [4]

**d** Identify which items of your test data are normal, erroneous and extreme. [3]

**8** This pseudocode algorithm inputs two non-zero numbers and a sign, and then performs the calculation shown by the sign. An input of zero for the first number terminates the process.

```
INPUT Number1, Number2, Sign
WHILE Number <> 0
    IF Sign = '+' THEN Answer ← Number1 + Number2 ENDIF
    IF Sign = '-' THEN Answer ← Number1 - Number2 ENDIF
    IF Sign = '*' THEN Answer ← Number1 * Number2 ENDIF
    IF Sign = '/' THEN Answer ← Number1 / Number2 ENDIF
    IF Sign <> '/' AND Sign <> '*' AND Sign <> '-' AND Sign <>
'+'
      THEN Answer ← 0
    ENDIF
    IF Answer <> 0 THEN OUTPUT Answer ENDIF
    INPUT Number1, Number2, Sign
ENDWHILE
```

**a** Complete the trace table for the input data:
5, 7, +, 6, 2, -, 4, 3, *, 7, 8, ?, 0, 0, /

| Number1 | Number2 | Sign | Answer | OUTPUT |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |

[3]

**b** Show how you could improve the algorithm written in pseudocode by writing an alternative type of conditional statement in pseudocode. [3]

*Cambridge IGCSE Computer Science (0478) Paper 22 Q3, June 2018*

**9** A programmer has written a routine to store the name, email address and password of a contributor to a website's discussion group.

**a** The programmer has chosen to verify the name, email address and password.
Explain why verification was chosen and describe how the programmer would verify this data. [4]

**b** The programmer has also decided to validate the email address and the password.
Describe validation checks that could be used. [2]

*Cambridge IGCSE Computer Science (0478) Paper 22 Q4, June 2018*